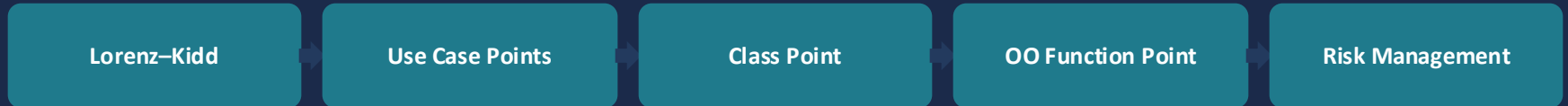


Unit – 2 (continued)

Object-Oriented Estimation Techniques & Risk Management

Lorenz–Kidd • Use Case Points • Class Point • OO Function Points • Risk Management



MCA – 212 • Object-Oriented Software Engineering

Dr Madhavi Dave

Learning Objectives

What students should be able to do after this module

01 • Lorenz & Kidd

- Estimate effort from key + support classes using interface multipliers and person-days per class.

02 • Use Case Points

- Compute UAW, UUCW, TCF and EF from actors, use cases and project factors to derive effort.

03 • Class Point

- Classify PDT/HIT/DMT/TMT classes by NEM & NSR and compute Total Unadjusted Class Points.

04 • OO Function Point

- Map classes and methods onto ILF/EIF/EI/EO/EQ and apply standard FPA complexity weights.

05 • Risk Management

- Identify, project and control project, technical and business risk using RE and RMMM plans.

- Apply every technique to the same running case study for direct comparison.
- Combine at least two independent estimates before committing to a schedule.

SECTION 1

Lorenz and Kidd Estimation Method

A class-counting approach to sizing OO projects — Object-Oriented Software Metrics (1994)

Why Lorenz and Kidd?

Turning an analysis-level class list into a defensible effort number

- Proposed by Mark Lorenz and Jeff Kidd in Object-Oriented Software Metrics: A Practical Guide (1994).
- Estimates effort directly from the number of classes identified during OO analysis — before detailed design exists.
- Distinguishes classes the requirements demand (key classes) from classes needed only to make the system run (support classes).
- Complements COCOMO/FP-style estimation: use it as one of two or more independent estimates, then reconcile.

Where it fits in the workflow

Requirements → Use Cases → Analysis Classes → Lorenz–Kidd effort estimate → Refine with Design classes (UCP / Class Point).

Best used the moment key classes are known but a full class diagram is not yet drawn.

The Lorenz–Kidd Procedure

Six steps from requirements to a person-day effort figure



1–2 • Baseline

- Develop a conventional estimate (decomposition / FP) and count use cases as a cross-check basis.

3 • Key Classes

- From the analysis model, count classes that come directly from the problem domain.

4–5 • Support Classes

- Categorize the interface type; multiply key classes by the matching multiplier to get support classes.

6 • Total Effort

- (Key + Support) classes × average person-days per class = total effort estimate.

Key Classes vs Support Classes

Two different sources of implementation effort

Key Classes

Classes derived directly from the problem domain and stated or implied by the requirements.

Identified during OO analysis — visible in the use-case and domain model.

Example (exam system): Student, Course, Examination, Question, Result.

Support Classes

Classes needed to realize the key classes as working software, but invisible to the end user's problem statement.

Cover the GUI, persistence/database access, communication, error handling, security.

Example: LoginWindow, DBConnector, SessionManager, Logger.

Step 4 — Interface Complexity Multiplier

Support classes scale with how demanding the user interface is

Interface Type	Description	Multiplier
No GUI	Batch / API-only, no interactive interface	2.0
Text-based UI	Character or command-line interface	2.25
GUI	Standard graphical interface (forms, menus)	2.5
Complex GUI	Rich, highly interactive graphical interface	3.0

Support Classes = Key Classes × Interface Multiplier

Step 6 — Work-Units per Class

Converting a class count into person-days

- Lorenz and Kidd suggest 15 to 20 person-days per class, covering design, coding and unit testing.
- The figure is an empirical average from industrial OO projects of the early 1990s — treat it as a starting point.
- Organizations should calibrate this productivity factor using their own historical project data whenever available.
- Simpler, well-understood domains trend toward 15 PD/class; novel or safety-critical domains trend toward 20+ PD/class.

Total Effort = (Key Classes + Support Classes) × Person-Days per Class

Lorenz–Kidd — Worked Example

Online Examination System

Given

Key classes identified = 22

Interface type = GUI → multiplier = 2.5

Productivity = 18 person-days / class

Working convention = 20 person-days / month

Calculation

Support classes = $22 \times 2.5 = 55$

Total classes = $22 + 55 = 77$

Effort = $77 \times 18 = 1,386$ person-days

$\approx 1,386 \div 20 = 69.3$ person-months

Reading the result

A single-developer team would take roughly 5.8 years; a 10-person team could target ~7 months, before adding schedule risk.

Cross-Checking the Estimate

Never trust a single number — triangulate

- Lorenz and Kidd recommend cross-checking the class-based estimate against use-case-based history.
- Multiply the number of use cases by the organization's average work-units per use case.

Class-based estimate

77 classes × 18 PD/class
= **1,386 person-days**

Use-case cross-check

18 use cases × 78 PD/use case
= **1,404 person-days**

Verdict

The two estimates differ by only ~1.3% — the class-based figure is credible and can be baselined.

Strengths & Limitations

What to trust, and what to calibrate

Strengths

Usable as soon as analysis (key) classes are known — no design model required.

Very quick to compute and easy to explain to non-technical stakeholders.

Grounded in published empirical data (person-days/class, multipliers).

Naturally supports a cross-check against use-case history.

Limitations

Multiplier and productivity figures need local calibration — 1990s data may not fit modern stacks.

"Key class" boundaries are subjective and vary between analysts.

Ignores use-case complexity, reuse, and team capability directly.

Coarser than Class Point or OO Function Point for later-stage estimates.

SECTION 2

Use Case Points (UCP) Method

Sizing directly from actors, use cases and project context factors

UCP — Method Overview

A natural fit for use-case-driven OO requirements



- Proposed by Gustav Karner (1993) for Rational/Objectory use-case driven projects.
- Weights actors and use cases by complexity, then adjusts for technical difficulty and team/environment maturity.
- Requires only use-case diagrams and brief use-case descriptions — available very early in the lifecycle.

Step 1 — Classify Actors (UAW)

Unadjusted Actor Weight

Actor Type	Description	Weight
Simple	Another system via a defined API	1
Average	Another system via a protocol, or interactive session	2
Complex	Human user interacting through a GUI	3

$$\text{UAW} = \Sigma (\text{number of actors of each type} \times \text{weight})$$

Step 2 — Classify Use Cases (UUCW)

Unadjusted Use Case Weight

Use Case Type	Transactions	Weight
Simple	≤ 3 transactions	5
Average	4 – 7 transactions	10
Complex	> 7 transactions	15

A "transaction" is one atomic exchange between actor and system (a request with its response).

$$\text{UUCW} = \Sigma (\text{number of use cases of each type} \times \text{weight})$$

Step 3 — Technical Complexity Factor (TCF)

13 technical factors rated 0 (irrelevant) to 5 (essential)

Factor	Weight	Factor	Weight
Distributed system	2.0	Reusable code	1.0
Response time / performance	1.0	Easy to install	0.5
End-user efficiency	1.0	Easy to use	0.5
Complex internal processing	1.0	Portable	2.0
Reusable code (target)	1.0	Easy to change	1.0
Concurrency	1.0	Security	1.0

$$\text{TCF} = 0.6 + 0.01 \times \sum (Ti)$$

Step 4 — Environmental Factor (EF)

8 factors on team and project maturity, rated 0 to 5

Factor	Weight	Effect
Familiarity with the development process	1.5	Positive
Application experience	0.5	Positive
OO experience of the team	1.0	Positive
Lead analyst capability	0.5	Positive
Motivation of the team	1.0	Positive
Stable requirements	2.0	Positive
Part-time staff on the project	-1.0	Negative
Difficult programming language	-1.0	Negative

$$EF = 1.4 - 0.03 \times \Sigma (E_i)$$

Putting It Together — UCP & Effort

From weighted counts to a person-hour estimate

$$\text{UCP} = (\text{UAW} + \text{UUCW}) \times \text{TCF} \times \text{EF}$$

$$\text{Effort (person-hours)} = \text{UCP} \times \text{Productivity Factor}$$

- Productivity factor is historically 20 person-hours per UCP; ranges from 15 (favorable EF) to 30 (unfavorable EF) are common.
- Karner's original guidance: if fewer than 2 of the 8 EF factors score below 3, use 20 hrs/UCP; otherwise recalibrate from history.

UCP — Worked Example

Library Management System

Actors & Use Cases

Actors: 2 simple, 1 average, 1 complex

$$\text{UAW} = 2(1) + 1(2) + 1(3) = 7$$

Use cases: 5 simple, 6 average, 3 complex

$$\text{UUCW} = 5(5) + 6(10) + 3(15) = 130$$

Factors

$$\Sigma \text{ Technical factors} = 32 \rightarrow \text{TCF} = 0.6 + 0.01(32) = 0.92$$

$$\Sigma \text{ Environmental factors} = 17 \rightarrow \text{EF} = 1.4 - 0.03(17) = 0.89$$

$$\text{UCP} = (7 + 130) \times 0.92 \times 0.89 \approx 112.2$$

Effort

$$\text{Effort} = 112.2 \times 20 \text{ hrs/UCP} \approx 2,244 \text{ person-hours} \approx 280 \text{ person-days (8 hr/day)} \approx 14 \text{ person-months (20 days/month)}.$$

SECTION 3

Class Point Method

Sizing an OO system directly from its UML class diagram — Costagliola et al. (2005)

Class Point — Overview

A Function-Point-style measure applied to classes, not files

- Extends the logic of Function Point Analysis to object-oriented design: classes are sized instead of logical files.
- Every class is classified into one of four system component types before it is measured.

PDT

- Problem Domain Type
— classes modeling real-world business entities.

HIT

- Human Interaction Type
— classes handling UI presentation and input.

DMT

- Data Management Type
— classes managing persistence & storage.

TMT

- Task Management Type
— classes controlling tasks & communication.

Class Point — Procedure

From class diagram to a size number



NEM

- Number of External Methods — public methods exposed by the class.

NSR

- Number of Services Requested — distinct external methods this class calls (coupling).

NOA (CP2 variant)

- Number of Attributes — used alongside NEM & NSR for a refined complexity rating.

Complexity Classification (CP1)

Based on NEM (methods) and NSR (services requested)

NEM \ NSR	0 – 3	4 – 7	≥ 8
0 – 3	Low	Low	Average
4 – 7	Low	Average	High
≥ 8	Average	High	High

Reading the table

A class with many public methods (high NEM) and many calls to other classes (high NSR) is structurally complex and costs more to build and test — regardless of which of the four component types it belongs to.

Class Point Weight Table

Complexity weight by component type

Type	Description	Low	Average	High
PDT	Problem domain	3	6	10
HIT	Human interaction	4	7	12
DMT	Data management	5	8	13
TMT	Task management	4	6	9

$$\text{TUCP} = \sum w_{ij} \times x_{ij} \quad (i = \text{class type}, j = \text{complexity level})$$

Class Point — Worked Example

Hospital Appointment System

Type	Low × w	Avg × w	High × w	Subtotal
PDT	$2 \times 3 = 6$	$3 \times 6 = 18$	$1 \times 10 = 10$	34
HIT	$1 \times 4 = 4$	$2 \times 7 = 14$	$0 \times 12 = 0$	18
DMT	$0 \times 5 = 0$	$1 \times 8 = 8$	$1 \times 13 = 13$	21
TMT	$1 \times 4 = 4$	$0 \times 6 = 0$	$0 \times 9 = 0$	4

TUCP = 34 + 18 + 21 + 4 = 77 class points

Next: apply the Technical Complexity Factor (TCF) to obtain the adjusted class points.

Class Point — Adjustment & Effort

From size to a person-hour estimate

- Like classic FPA, the raw count is adjusted using General System Characteristics (GSCs) rated 0–5 each.

$$\text{CFP} = \text{TUCP} \times (0.65 + 0.01 \times \Sigma \text{GSC})$$

$$\text{Effort} = \text{CFP} \times \text{Productivity Factor (person-hours / point)}$$

Given

TUCP = 77

$\Sigma \text{GSC} = 48 \rightarrow \text{VAF} = 0.65 + 0.48 = 1.13$

Productivity = 6 person-hrs / point

Result

$\text{CFP} = 77 \times 1.13 = 87.0$

$\text{Effort} = 87.0 \times 6 \approx 522 \text{ person-hours}$

$\approx 65 \text{ person-days (8 hr/day)}$

SECTION 4

Object-Oriented Function Point (OOFP)

Adapting classic IFPUG Function Point Analysis to an object model

OOFP — Mapping OO Constructs to FP Types

Reusing decades of Function Point calibration data for OO systems

OO Construct	FP Function Type
Class with its own persistent attributes	Internal Logical File (ILF)
Referenced class owned by another system	External Interface File (EIF)
Method that creates / updates / deletes data	External Input (EI)
Method that derives & outputs data	External Output (EO)
Method that simply retrieves data	External Inquiry (EQ)

OOFP — Counting Rules

Reusing DET / FTR from classic FPA

DET

- Data Element Type $\approx$ number of attributes (for a class/ILF) or parameters (for a method/transaction).

RET / FTR

- Record Element Type / File Type Referenced $\approx$ number of associated classes touched by the method or file.

Complexity

- DET and RET/FTR counts are looked up in the standard IFPUG matrices to rate each item Low, Average or High.

Why reuse FPA?

Organizations that already maintain historical Function Point productivity data (hrs/FP) can plug OOFP counts straight into existing estimation models — no new calibration needed.

Standard IFPUG Complexity Weights

Applied unchanged to OO-mapped function types

Function Type	Low	Average	High
ILF / EIF (classes)	7	10	15
EI (create/update methods)	3	4	6
EO / EQ (output/query methods)	4	5	7

$$\text{UFP} = \Sigma (\text{ILF+EIF}) + \Sigma (\text{EI}) + \Sigma (\text{EO+EQ})$$

OOFP — Worked Example

Payroll Processing System

Function Type	Count breakdown	Subtotal
ILF (classes)	2 Low(7) + 1 Avg(10) + 1 High(15)	39
EI (update methods)	5 Low(3) + 3 Avg(4) + 2 High(6)	39
EO (report methods)	2 Avg(5) + 2 High(7)	24
EQ (query methods)	4 Low(4) + 2 Avg(5)	26

UFP = 39 + 39 + 24 + 26 = 128 unadjusted function points

OOFP — Adjustment & Effort

Value Adjustment Factor and final effort

$$\text{VAF} = 0.65 + 0.01 \times \Sigma \text{GSC} \quad (14 \text{ General System Characteristics, } 0\text{--}5 \text{ each})$$

Given

UFP = 128

$\Sigma \text{GSC} = 45 \rightarrow \text{VAF} = 0.65 + 0.45 = 1.10$

Productivity = 8 person-hrs / FP

Result

$\text{AFP} = 128 \times 1.10 = 140.8$

$\text{Effort} = 140.8 \times 8 \approx 1,126 \text{ hrs}$

$\approx 140.8 \text{ person-days (8 hr/day)}$

Note

OOFP tends to run larger than Class Point for the same system because every public method is counted individually as a transaction, not aggregated per class.

SECTION 5

Risk Management

Identify uncertainty early, and control it deliberately

Why Risk Management?

A risk is a potential problem — it might happen, it might not

- Risk management is proactive: identify, assess and plan a response before a problem occurs — not after.
- Every estimate carries uncertainty; risk management makes that uncertainty explicit and manageable.
- Unmanaged risk is the leading cause of schedule slip, budget overrun and quality failure in software projects.

Reactive vs Proactive

Reactive ("fire-fighting"): the team reacts only when a risk becomes a real problem — costly and stressful.

Proactive: risks are identified, their probability and impact assessed, and a mitigation/contingency plan is ready before they occur.

Categories of Risk

Where the threat comes from

Project Risks

- Threaten the project plan: schedule slip, budget overrun, staffing/resource shortfall.

Technical Risks

- Threaten quality and timeliness: design flaws, implementation difficulty, integration and verification problems.

Business Risks

- Threaten the viability of the product: market risk, strategic risk, management risk, loss of budget/support.

Also classified by predictability

Known risks: identifiable after careful review (e.g., unrealistic delivery date).

Predictable risks: extrapolated from past project experience (e.g., staff turnover).

Unpredictable risks: can occur but are extremely difficult to identify in advance.

Risk Identification Checklist

Standard categories to probe during risk identification

Product size

- Size estimates, reuse, database size, users to support

Business impact

- Effect on company revenue, visibility to senior management

Customer characteristics

- Communication, sophistication, agreement on requirements

Process definition

- Maturity and consistency of the development process used

Development environment

- Tools available and their integration/reliability

Technology to be built

- Novelty of the technology to the organization

Staff size & experience

- Technical and project experience of assigned staff

Risk Projection (Assessment)

Turning a list of risks into numbers



$$\text{Risk Exposure (RE)} = \text{Probability (P)} \times \text{Consequence (C)}$$

- P — likelihood the risk occurs, expressed 0.0 – 1.0 (or a qualitative scale converted to a number).
- C — cost/consequence to the project if it does occur, in effort (person-days), money, or schedule (days).

Impact (Consequence) Categories

A common qualitative scale for C

Category	Definition
Catastrophic	Threatens the project or the organization itself
Critical	Major delay, cost overrun, or quality shortfall
Marginal	Minor, manageable delay or impact
Negligible	Nuisance-level; little practical effect

Risk Table — Worked Example

Sorted by risk exposure, highest first

Risk	P	Impact (PD)	RE = P×C
Requirements change late	0.6	40	24
Key developer leaves	0.25	60	15
Third-party API delayed	0.4	25	10
Performance target missed	0.2	35	7

Reading the table

Sorting by RE focuses attention where it matters: "Requirements change late" carries the greatest exposure (24 person-days) even though its impact alone is smaller than "Key developer leaves".

RMMM — Mitigation, Monitoring, Management

Turning an assessed risk into a controlled one

Mitigation

- Proactive steps taken before the risk occurs, to reduce its probability or impact.

Monitoring

- Ongoing tracking of factors/indicators that signal the risk is becoming more (or less) likely.

Management

- The contingency plan executed if the risk does become a real problem.

Example — "Requirements change late"

Mitigation: freeze scope after SRS sign-off; prioritize backlog with MoSCoW.

Monitoring: track change-request rate weekly against a threshold.

Management: reserve 10% schedule contingency; route changes through a change-control board.

The RMMM Plan

What every documented risk should record

- Risk ID, one-line description, and category (project / technical / business).
- Probability (P) and consequence/impact (C), with the resulting Risk Exposure (RE).
- Mitigation strategy — what is done now to reduce P or C.
- Monitoring indicators — what signal is watched, and how often.
- Contingency plan — what happens if the risk is realized despite mitigation.
- Owner and trigger/threshold — who is accountable, and what condition activates the contingency.

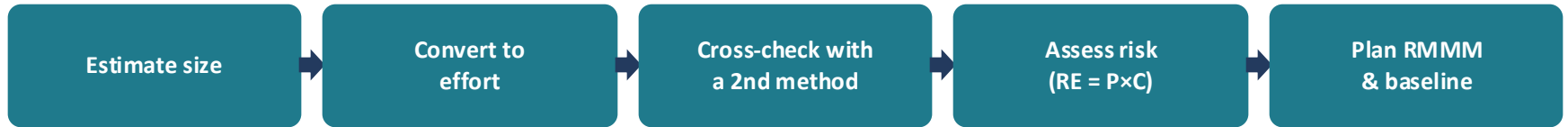
Comparing the Four Estimation Methods

Pick the method that matches what you know right now

Method	Primary Input	Best Used When	Core Formula
Lorenz–Kidd	Key classes + interface type	Analysis classes known; quick early estimate	$(\text{Key} + \text{Support}) \times \text{PD} / \text{class}$
Use Case Points	Actors + use cases	Use-case-driven requirements available	$(\text{UAW} + \text{UUCW}) \times \text{TCF} \times \text{EF}$
Class Point	UML class diagram (NEM/NSR)	Detailed class diagram is ready	$\sum w \times x$
OO Function Point	Classes mapped to FP types	Organization already has FPA history	$\sum (\text{EI} + \text{EO} + \text{EQ} + \text{ILF} + \text{EIF})$

Key Takeaways

One engineering story, from class count to controlled risk



- No single estimation method is "correct" — always triangulate with at least two independent techniques.
- Estimates should be refined progressively as the class model, use cases and design mature.
- Express every estimate as a range or with explicit assumptions, not a single false-precision number.
- Pair every size/effort estimate with a risk assessment (RE) and an RMMM plan before committing to a schedule.